

# MI AEC API

---

**Version 1.1**

© 2020 SigmaStar Technology Corp. All rights reserved.

SigmaStar Technology makes no representations or warranties including, for example but not limited to, warranties of merchantability, fitness for a particular purpose, non-infringement of any intellectual property right or the accuracy or completeness of this document, and reserves the right to make changes without further notice to any products herein to improve reliability, function or design. No responsibility is assumed by SigmaStar Technology arising out of the application or use of any product or circuit described herein; neither does it convey any license under its patent rights, nor the rights of others.

SigmaStar is a trademark of SigmaStar Technology Corp. Other trademarks or names herein are only for identification purposes only and owned by their respective owners.

## REVISION HISTORY

| Revision No. | Description       | Date       |
|--------------|-------------------|------------|
| 1.0          | ● Initial release | 09/08/2020 |
| 1.1          | ● Modified API    | 10/15/2020 |

## 目录

|                           |           |
|---------------------------|-----------|
| <b>REVISION HISTORY</b>   | <b>i</b>  |
| 目录                        | ii        |
| <b>1. 概述</b>              | <b>1</b>  |
| 1.1. 算法说明                 | 1         |
| 1.2. 关键词说明                | 1         |
| 1.3. 注意                   | 2         |
| <b>2. API 参考</b>          | <b>3</b>  |
| 2.1. 功能模块 API             | 3         |
| 2.2. IaaAec_GetBufferSize | 3         |
| 2.3. IaaAec_Init          | 4         |
| 2.4. IaaAec_Config        | 5         |
| 2.5. IaaAec_Reset         | 5         |
| 2.6. IaaAec_Free          | 6         |
| 2.7. IaaAec_Run           | 7         |
| 2.8. IaaAec_GetLibVersion | 11        |
| <b>3. AEC 数据类型</b>        | <b>13</b> |
| 3.1. AEC 模块相关数据类型定义       | 13        |
| 3.2. AudioAecInit         | 13        |
| 3.3. AudioAecConfig       | 14        |
| 3.4. IAA_AEC_SAMPLE_RATE  | 15        |
| 3.5. IAA_AEC_BOOL         | 16        |
| 3.6. AEC_HANDLE           | 17        |
| <b>4. 错误码</b>             | <b>18</b> |

# 1. 概述

## 1.1. 算法说明

回声消除 (Acoustic Echo Cancellation, 简称 AEC)，是一种用于抑制远程回声的功能。

回声常见于会议系统、楼宇对讲、安防监控等场景，当远程声音从喇叭播放出来后，麦克风在非常小的延时后，重新采集喇叭播放出来的声音并传输回远程，这个延迟的原始信号将导致远程听到回声。

## 1.2. 关键词说明

- Far-end signal (Sin)

设备扬声器的信号源，远程传来的信号。

- Near-end signal (Rin)

设备麦克风录到的信号，此信号可能包含声学回声(Acoustic echo)和近端语者(Near-end talker)的信号

- Acoustic echo (AE)

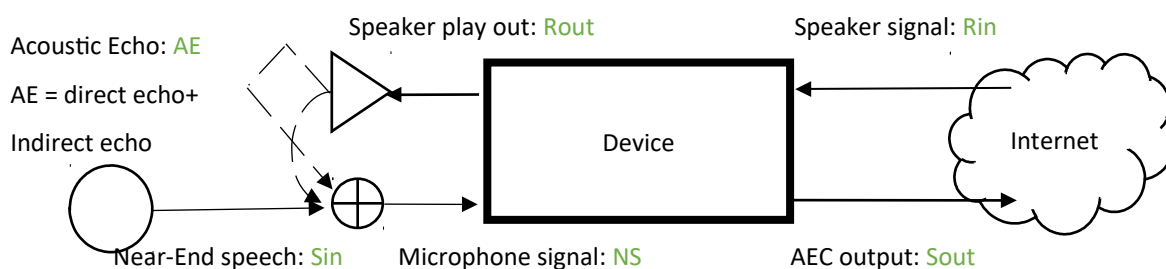
从设备扬声器播放的声音通过直接或间接路径传回麦克风，产生回声；远程语者在说话后会在延迟某段时间后听到自己的回声，此回声称为声学回声。

- Single talk

只有设备端的喇叭播放远程信号( $\text{Sin} = \text{AE} + \text{NS}$  当  $\text{NS} = 0$ )

- Double talk

设备端喇叭播放的同时近端语者也对着麦克风说话( $\text{Sin} = \text{AE} + \text{NS}$  当  $\text{NS} \neq 0$ )



## 1.3. 注意

为方便调试和确认算法效果，需要用户应用自行实现替换算法参数和抓取音频数据的逻辑。

## 2. API 参考

### 2.1. 功能模块 API

| API 名                                | 功能                 |
|--------------------------------------|--------------------|
| <a href="#">IaaAec_GetBufferSize</a> | 获取 Aec 算法运行需要的内存大小 |
| <a href="#">IaaAec_Init</a>          | 初始化 Aec 算法         |
| <a href="#">IaaAec_Config</a>        | 配置 Aec 算法          |
| <a href="#">IaaAec_Reset</a>         | 重新初始化 Aec 算法       |
| <a href="#">IaaAec_Free</a>          | 释放 Aec 算法资源        |
| <a href="#">IaaAec_Run</a>           | Aec 算法处理           |
| <a href="#">IaaAec_GetLibVersion</a> | 获取 Aec 算法的版本信息     |

### 2.2. IaaAec\_GetBufferSize

#### ➤ 功能

获取 Aec 算法运行所需要的内存大小。

#### ➤ 语法

```
unsigned int IaaAec_GetBufferSize(void);
```

#### ➤ 形参

| 参数名称 | 描述 | 输入/输出 |
|------|----|-------|
|------|----|-------|

#### ➤ 返回值

返回值为 Aec 算法运行所需要的内存大小

#### ➤ 依赖

- 头文件: AudioAecProcess.h

- 库文件: libAEC\_LINUX.so/ libAEC\_LINUX.a

※ 注意

该接口仅返回需要的内存大小，申请和释放内存的动作需应用来处理。

➤ 举例

无。

## 2.3. IaaAec\_Init

➤ 功能

初始化 Aec 算法需要的内存。

➤ 语法

[AEC\\_HANDLE](#) IaaAec\_Init(char\* working\_buffer\_address, [AudioAecInit](#) \* aec\_init);

➤ 形参

| 参数名称                   | 描述              | 输入/输出 |
|------------------------|-----------------|-------|
| working_buffer_address | Aec 算法使用的内存地址   | 输入    |
| aec_init               | Aec 算法的初始化结构体指针 | 输入    |

➤ 返回值

| 返回值 | 结果                        |
|-----|---------------------------|
| 0   | 成功                        |
| 非 0 | 失败，参照 <a href="#">错误码</a> |

➤ 依赖

- 头文件: AudioAecProcess.h
- 库文件: libAEC\_LINUX.so/ libAEC\_LINUX.a

※ 注意

- Aec 算法仅支持 8K/16K 采样率。

➤ 举例

无。

## 2.4. IaaAec\_Config

### ➤ 功能

配置 Aec 算法。

### ➤ 语法

```
ALGO_AEC_RET IaaAec_Config(AEC\_HANDLE handle, AudioAecConfig *aec_config);
```

### ➤ 形参

| 参数名称       | 描述               | 输入/输出 |
|------------|------------------|-------|
| handle     | Aec 算法 handle    | 输入    |
| aec_config | Aec 算法配置参数的结构体指针 | 输入    |

### ➤ 返回值

| 返回值 | 结果                        |
|-----|---------------------------|
| 0   | 成功                        |
| 非 0 | 失败，参照 <a href="#">错误码</a> |

### ➤ 依赖

- 头文件: AudioAecProcess.h
- 库文件: libAEC\_LINUX.so/ libAEC\_LINUX.a

### ※ 注意

无。

### ➤ 举例

无。

## 2.5. IaaAec\_Reset

### ➤ 功能

重新初始化 Aec 算法的内存。

## ➤ 语法

```
AEC\_HANDLE IaaAec_Reset(char* working_buffer_address, AudioAecInit * aec_init);
```

## ➤ 形参

| 参数名称                   | 描述              | 输入/输出 |
|------------------------|-----------------|-------|
| working_buffer_address | Aec 算法所使用的内存地址  | 输入    |
| aec_init               | Aec 算法的初始化结构体指针 | 输入    |

## ➤ 返回值

| 返回值 | 结果                        |
|-----|---------------------------|
| 0   | 成功                        |
| 非 0 | 失败，参照 <a href="#">错误码</a> |

## ➤ 依赖

- 头文件: AudioAecProcess.h
- 库文件: libAEC\_LINUX.so/ libAEC\_LINUX.a

## ※ 注意

- 针对同一个算法 handle（同一内存地址）重新初始化 Aec 算法（即重新设置采样率和近远程的通道数）
- 重新初始化 Aec 算法后，需要重新配置

## ➤ 举例

无。

## 2.6. IaaAec\_Free

## ➤ 功能

释放 Aec 算法的资源

## ➤ 语法

```
ALGO_AEC_RET IaaAec_Free(AEC\_HANDLE handle);
```

## ➤ 形参

| 参数名称   | 描述            | 输入/输出 |
|--------|---------------|-------|
| handle | Aec 算法 handle | 输入    |

➤ 返回值

| 返回值 | 结果                        |
|-----|---------------------------|
| 0   | 成功                        |
| 非 0 | 失败，参照 <a href="#">错误码</a> |

➤ 依赖

- 头文件：AudioAecProcess.h
- 库文件：libAEC\_LINUX.so/ libAEC\_LINUX.a

※ 注意

- 必须先调用 IaaAec\_Free，再释放供 Aec 算法所使用的内存。

➤ 举例

无。

## 2.7. IaaAec\_Run

➤ 功能

Aec 算法处理

➤ 语法

```
ALGO_AEC_RET IaaAec_Run(AEC\_HANDLE handle, short* pss_audio_near_end, short* pss_audio_far_end);
```

## ➤ 形参

| 参数名称               | 描述        | 输入/输出 |
|--------------------|-----------|-------|
| handle             | 算法 handle | 输入    |
| pss_audio_near_end | 近端数据指针    | 输入/输出 |
| pss_audio_far_end  | 远程数据指针    | 输入    |

## ➤ 返回值

| 返回值 | 结果                        |
|-----|---------------------------|
| 0   | 成功                        |
| 非 0 | 失败，参照 <a href="#">错误码</a> |

## ➤ 依赖

- 头文件: AudioAecProcess.h
- 库文件: libAEC\_LINUX.so/ libAEC\_LINUX.a

## ※ 注意

- 近远程的数据量必须和调用 IaaAec\_Init 时设定的 point\_number（一次 IaaAec\_Run 所需要的采样点数）相对应。
- AEC 后结果由 pss\_audio\_near\_end 输出

## ➤ 举例

```

1. #include <stdio.h>
2. #include <string.h>
3. #include <sys/time.h>
4. #include <stdlib.h>
5.
6. #include "AudioAecProcess.h"
7.
8. /* 0:Fixed input file 1:User input file */
9. #define IN_PARAMETER 1
10.
11. float AVERAGE_RUN(int a)
12. {
13.     static unsigned int num = 0;
14.     static float avg = 0;
15.     if(0 == num)
16.         avg = 0;
17.     num++;
18.     avg = avg + ((float)a - avg) / ((float)num);
19.     return avg;
20. }
21.

```

```

22. unsigned int _OsCounterGetMs(void)
23. {
24.     struct timeval t1;
25.     gettimeofday(&t1, NULL);
26.     unsigned int T = ((1000000 * t1.tv_sec) + t1.tv_usec) / 1000;
27.     return T;
28. }
29.
30. int main(int argc, char *argv[])
31. {
32.     short in_output[1024];
33.     short input_far[1024];
34.     char src_file1[128] = {0};
35.     char src_file2[128] = {0};
36.     char dst_file[128] = {0};
37.     unsigned int workingBufferSize;
38.     char *workingBuffer = NULL;
39.     int counter = 0;
40.     ALGO_AEC_RET ret;
41.     int tempSize;
42.     unsigned int T0, T1;
43.     float avg;
44.     FILE *fpInFar, *fpOut, *fpInNear;
45.     AudioAecInit aec_init;
46.     AudioAecConfig aec_config;
47.     AEC_HANDLE handle;
48.     /******User change section start******/
49.     unsigned int supMode_band[6] = {20,40,60,80,100,120};
50.     unsigned int supMode[7] = {4,4,4,4,4,4,4};
51.     aec_init.point_number = 128;
52.     aec_init.nearend_channel = 1;
53.     aec_init.farend_channel = 1;
54.     aec_init.sample_rate = IAA_AEC_SAMPLE_RATE_16000;
55.     aec_config.delay_sample = 0;
56.     aec_config.comfort_noise_enable = IAA_AEC_FALSE;
57.     /******User change section end******/
58.     memcpy(&(aec_config.suppression_mode_freq[0]), supMode_band, sizeof(supMode_band));
59.     memcpy(&(aec_config.suppression_mode_intensity[0]), supMode, sizeof(supMode));
60.     //(1)IaaAec_GetBufferSize
61.     workingBufferSize = IaaAec_GetBufferSize();
62.     workingBuffer = (char*)malloc(workingBufferSize);
63.     if(NULL == workingBuffer)
64.     {
65.         printf("workingBuffer malloc failed !\n");
66.         return -1;
67.     }
68.     printf("workingBuffer malloc success !\n");
69.     //(2)IaaAec_Init
70.     handle = IaaAec_Init(workingBuffer, &aec_init);
71.     if (NULL == handle)
72.     {
73.         printf("AEC init failed !\r\n");
74.         return -1;
75.     }
76.     printf("AEC init success !\r\n");
77.     //(3)IaaAec_Config
78.     ret = IaaAec_Config(handle, &aec_config);
79.     if(ret)
80.     {
81.         printf("IaaAec_Config failed !\n");
82.         return -1;
83.     }

```

```

84.     printf("IaaAec_Config succeed !\n");
85.
86. #if IN_PARAMETER
87.     if(argc < 4)
88.     {
89.         printf("Please enter the correct parameters!\n");
90.         return -1;
91.     }
92.     sscanf(argv[1], "%s", src_file1);
93.     sscanf(argv[2], "%s", src_file2);
94.     sscanf(argv[3], "%s", dst_file);
95. #else
96.     sprintf(src_file1, "%s", "./farend.wav");
97.     sprintf(src_file2, "%s", "./nearend.wav");
98.     if(argc < 2)
99.     {
100.        printf("Please enter the correct parameters!\n");
101.        return -1;
102.    }
103.    sscanf(argv[1], "%s", dst_file);
104. #endif
105.
106.    fpInFar = fopen(src_file1, "rb");
107.    if(NULL == fpInFar)
108.    {
109.        printf("src_file1 open failed !\n");
110.        return -1;
111.    }
112.    printf("src_file1 open succeed !\n");
113.    fpInNear = fopen(src_file2, "rb");
114.    if(NULL == fpInNear)
115.    {
116.        printf("src_file2 open failed !\n");
117.        return -1;
118.    }
119.    printf("src_file2 open succeed !\n");
120.    fpOut = fopen(dst_file, "wb");
121.    if(NULL == fpOut)
122.    {
123.        printf("dst_file open failed !\n");
124.        return -1;
125.    }
126.    printf("dst_file open succeed !\n");
127.    #if 1
128.        fread(in_output, sizeof(char), 44, fpInNear); // Remove the 44 bytes header
129.        fwrite(in_output, sizeof(char), 44, fpOut); // New file add header
130.        fread(input_far, sizeof(char), 44, fpInFar); // Remove the 44 bytes header
131.    #endif
132.    tempSize = aec_init.point_number * aec_init.nearend_channel;
133.    while(fread(in_output, sizeof(short), tempSize, fpInNear))
134.    {
135.        tempSize = aec_init.point_number * aec_init.farend_channel;
136.        fread(input_far, sizeof(short), tempSize, fpInFar);
137.        counter++;
138.        T0 = (long)_OsCounterGetMs();
139.        //(4)IaaAec_Run
140.        ret = IaaAec_Run(handle, in_output, input_far);
141.        T1 = (long)_OsCounterGetMs();
142.        avg = AVERAGE_RUN(T1 - T0);
143.        if(0 == counter % 100)
144.        {
145.            printf("counter = %d\n", counter);

```

```

146.         printf("current time = %f\n", (float)counter * aec_init.point_number / aec_ini
t.sample_rate);
147.         printf("process time = %lu(ms)\t", (long)(T1 - T0));
148.         printf("AVG is %.2f ms\n", avg);
149.     }
150.     if(ret < 0)
151.     {
152.         printf("IaaAec_Run failed !\n");
153.         break;
154.     }
155.     tempSize = aec_init.point_number * aec_init.nearend_channel;
156.     fwrite(in_output, sizeof(short), tempSize, fpOut);
157. }
158. //(5)IaaAec_Free
159. IaaAec_Free(handle);
160. fclose(fpInFar);
161. fclose(fpInNear);
162. fclose(fpOut);
163. free(workingBuffer);
164. printf("AEC end !\n");
165.
166. return 0;
167. }

```

## 2.8. IaaAec\_GetLibVersion

### ➤ 功能

获取 Aec 算法的版本信息

### ➤ 语法

```

ALGO_AEC_RET IaaAec_GetLibVersion(unsigned short *ver_year,
                                   unsigned short *ver_date,
                                   unsigned short *ver_time);

```

### ➤ 形参

| 参数名称     | 描述              | 输入/输出 |
|----------|-----------------|-------|
| ver_year | Aec 算法库文件编译时的年份 | 输出    |
| ver_date | Aec 算法库文件编译时的日期 | 输出    |
| ver_time | Aec 算法库文件编译时的时间 | 输出    |

### ➤ 返回值

| 返回值 | 结果                        |
|-----|---------------------------|
| 0   | 成功                        |
| 非 0 | 失败，参照 <a href="#">错误码</a> |

➤ 依赖

- 头文件: AudioAecProcess.h
- 库文件: libAEC\_LINUX.so/ libAEC\_LINUX.a

※ 注意

无。

➤ 举例

无。

## 3. AEC 数据类型

### 3.1. AEC 模块相关数据类型定义

| 数据类型                | 定义                 |
|---------------------|--------------------|
| AudioAecInit        | Aec 算法初始化参数数据结构体类型 |
| AudioAecConfig      | Aec 算法配置参数数据结构体类型  |
| IAA_AEC_SAMPLE_RATE | Aec 算法支持的采样率类型     |
| IAA_AEC_BOOL        | Aec 算法的布尔类型        |
| AEC_HANDLE          | Aec 算法句柄类型         |

### 3.2. AudioAecInit

➤ 说明

定义 Aec 算法的初始化参数结构体。

➤ 定义

```
typedef struct
{
    unsigned int point_number;
    unsigned int nearend_channel;
    unsigned int farend_channel;
    IAA_AEC_SAMPLE_RATE sample_rate;
}AudioAecInit;
```

➤ 成员

| 成员名称            | 描述               |
|-----------------|------------------|
| point_number    | Aec 算法处理一次的采样点数  |
| nearend_channel | 远程的信道数（即麦克风的信道数） |
| farend_channel  | 近端的信道数（即喇叭的通道数）  |
| sample_rate     | Aec 算法处理的采样率     |

※ 注意事项

无

➤ 相关数据类型及接口

[IaaAec\\_Init](#)、[IaaAec\\_Reset](#)

### 3.3. AudioAecConfig

➤ 说明

定义 Aec 算法的配置参数结构体。

➤ 定义

```
typedef struct
{
    IAA_AEC_BOOL comfort_noise_enable;
    short delay_sample;
    unsigned int suppression_mode_freq[6];
    unsigned int suppression_mode_intensity[7];
}AudioAecConfig;
```

➤ 成员

| 成员名称                  | 描述                                                                                                         |
|-----------------------|------------------------------------------------------------------------------------------------------------|
| comfort_noise_enable  | 在 AEC 后加入舒适噪声，避免回声消除后部分段落静音，让用户有断线的感觉。<br>若 AEC 后有开启 Noise reduction，建议开启 comfort noise，帮助 NR 收敛噪声。        |
| delay_sample          | 左右声道间的回声延迟样本<br>当 Stereo cost down 开启时，可以根据麦克风摆放位置先行设定样本延迟数，减少双声道所需运算量。<br>默认为 0，单位是 sample 数<br>取值范围:-9-9 |
| suppression_mode_freq | AEC 处理的频段划分，将不同取样率所能解析的最高频率等分                                                                              |

| 成员名称                       | 描述                                                        |
|----------------------------|-----------------------------------------------------------|
|                            | 成 128 等分，设定六个值可切出七个段落设定不同 AEC 强度。<br>取值范围：1-127           |
| suppression_mode_intensity | 配合 suppression_mode_freq 所切割出来的七个段落分别设定七种强度。<br>取值范围：0-25 |

#### ※ 注意事项

- **comfort\_noise\_enable**

使能该参数后，AEC 算法会在没有声音时添加一些舒适噪声。当客户对稳定的底噪有要求或 AEC 消得太干净导致 NR 收敛时间太长时，可使能该功能。

- **delay\_sample**

由于麦克风和喇叭的放置位置、麦克风间的距离等会造成左右声道接收到回声的时间点不一致，两个声道之间回声延迟有差异。此值表示左声道比右声道提早多少个采样点收到回声。可以调整此值来对齐左右声道的资料，设置时，请以左声道作为参考。例如左声道的回声比右声道的回声快 4 个样本，则 s16DelaySample 设置为 4，反之设置为-4。

- **suppression\_mode\_freq**

该数组将当前采样率的最高频率分成 7 个频段来处理。以下数组元素和频率范围的转换公式： $f_i$ ，数组要求每一个元素都必须比前一个元素大。

- **suppression\_mode\_intensity**

该数组代表各个频段的回声消除强度，各个元素与 u32AecSupfreq 划分出来的 7 个频段一一对应。强度越大，消除的细节越多，声音越不自然。

#### ➤ 相关数据类型及接口

[IaaAec\\_Config](#)

## 3.4. IAA\_AEC\_SAMPLE\_RATE

#### ➤ 说明

AEC 算法支持的采样率类型。

#### ➤ 定义

```
typedef enum
{
    IAA_AEC_SAMPLE_RATE_8000 = 8000 ,
```

```
    IAA_AEC_SAMPLE_RATE_16000 = 16000 ,  
}IAA_AEC_SAMPLE_RATE;
```

## ➤ 成员

| 成员名称                      | 描述      |
|---------------------------|---------|
| IAA_AEC_SAMPLE_RATE_8000  | 8K 采样率  |
| IAA_AEC_SAMPLE_RATE_16000 | 16K 采样率 |

## ※ 注意事项

无

## ➤ 相关数据类型及接口

[AudioAecInit](#)

### 3.5. IAA\_AEC\_BOOL

## ➤ 说明

定义 Aec 算法内部使用的布尔类型。

## ➤ 定义

```
typedef enum
{
    IAA_AEC_TRUE = 1,
    IAA_AEC_FALSE = 0,
}IAA_AEC_BOOL;
```

## ➤ 成员

| 成员名称          | 描述    |
|---------------|-------|
| IAA_AEC_TRUE  | True  |
| IAA_AEC_FALSE | False |

## ※ 注意事项

无

## ➤ 相关数据类型及接口

[AudioAecConfig](#)

### 3.6. AEC\_HANDLE

➤ 说明

定义 Aec 算法的句柄类型。

➤ 定义

```
typedef void* AEC_HANDLE;
```

➤ 成员

| 成员名称 | 描述 |
|------|----|
|------|----|

※ 注意事项

无

➤ 相关数据类型及接口

[IaaAec\\_Init](#)、[IaaAec\\_Config](#)、[IaaAec\\_Reset](#)、[IaaAec\\_Free](#)、[IaaAec\\_Run](#)

## 4. 错误码

AEC API 错误码如表下所示：

| 错误码        | 宏定义                                | 描述             |
|------------|------------------------------------|----------------|
| 0x00000000 | ALGO_AEC_RET_SUCCESS               | AEC 运行成功       |
| 0x10000401 | ALGO_AEC_RET_INIT_ERROR            | AEC 尚未初始化      |
| 0x10000402 | ALGO_AEC_RET_INVALID_HANDLE        | HANDLE 无效      |
| 0x10000403 | ALGO_AEC_RET_INVALID_SAMPLE_RATE   | 取样频率不支持        |
| 0x10000404 | ALGO_AEC_RET_INVALID_POINT_NUMBER  | 每帧点数不支持        |
| 0x10000405 | ALGO_AEC_RET_INVALID_CHANNEL       | 通道数不支持         |
| 0x10000406 | ALGO_AEC_RET_INVALID_SUP_BAND      | AEC 抑制频带参数设置无效 |
| 0x10000407 | ALGO_AEC_RET_INVALID_SUP_MODE      | AEC 抑制强度参数设置无效 |
| 0x10000408 | ALGO_AEC_RET_INVALID_COMFORT_NOISE | AEC 舒适噪声参数设置无效 |
| 0x10000409 | ALGO_AEC_RET_INVALID_DELAY_SAMPLE  | AEC 延迟样本数设置无效  |
| 0x10000410 | ALGO_AEC_RET_API_CONFLICT          | 其他 API 正在运行    |

| 错误码        | 宏定义                          | 描述          |
|------------|------------------------------|-------------|
| 0x10000411 | ALGO_AEC_RET_INVALID_CALLING | 呼叫 API 顺序错误 |